Visualizing Responsibility Distribution in the Linux Kernel Codebase

Guilherme Luiz Pereira de Almeida, Rodrigo Dassie Paneto
Supervisor: Arthur Pilone Maia da Silva
Co-supervisor: Prof. Dr. Paulo Meirelles

1 Introduction

1.1 Context and motivation

The open-source community has been, for as long as it has existed, a prominent facet of the software zeitgeist. Throughout the years, we have seen more and more software projects created and maintained following this format. Indeed, the ability to create such large-scale products backed by a global array of contributors proved to be quite a successful formula, as research indicates that the labor replacement value of open-source software is so significant that firms would need to spend over three times more on software development if these resources did not exist [4] .

Whether it be the numerous contributions of the GNU project, or the Linux kernel, the truth is that the FLOSS community has achieved impressive results. Unlike traditional proprietary software, which often faces diminishing returns as complexity increases, the Linux kernel has demonstrated a unique pattern, expanding its codebase and contributor base simultaneously for decades [5] .

As with any large software codebase, open-source projects require robust organizational cohesion. Sustaining a practical hierarchy of contributors while also ensuring that no person is overburdened and that a large part of the knowledge does not end up concentrated in a single individual is not an easy task, especially in the long term. Many efforts were made to better map these kinds of issues inside a codebase, such as:

- The Dashboard for Unified Kernel Statistics (DUKS) is "an analytical platform providing insights into Linux Kernel development cycles, contributor patterns, and collaborative dynamics within one of the world's largest open-source projects" [7], envisioned as "a cornerstone open-access utility to support analyses on the Linux kernel evolution and maintenance" [6].
- The Developer Rivers, "a timeline-based visualization technique that shows how developers work on software modules. The flow of developers' activity is visualized by a river metaphor: activities are transferred between modules represented as rivers" [3].

- The DENIM project, a "tool that enables users to identify data access points in microservices and visualize them in an interactive treemap" [1].

All these tools aim to simplify the process of analyzing large codebases. They also help manage complex work hierarchies. As previously stated, managing these hierarchies is one of the biggest challenges when dealing with open source software.

However, there is still a lack of tools for web visualizations of responsibility distributions on large codebases such as the Linux Kernel. The sheer volume of contribution data and the size of the repository itself make concise visualization with good rendering performance difficult. Viewing this challenge as an opportunity, this work will focus on doing exactly that. It will provide a web visualization using a Tree-Map that aims to be both performant and user-friendly.

2 Proposal

2.1 Objectives

Aiming to provide an interactive software visualization tool that efficiently depicts the complete codebase hierarchy of large FLOSS codebases, the objectives of this work are to:

- Evaluate the previously developed tools and how they could be integrated to our work.
- Develop an improved interactive Tree-Map [2] visualization of the Kernel, displaying the responsibility distribution of the kernel codebase.
- Make the implementation more efficient, fluid, and user-friendly, removing the stuttering present in some of the old visualizations.

2.2 Methodology

The methodology is structured into three primary phases:

- An initial evaluation of the existing Tree-Map implementations by Silva and André to identify specific bottlenecks in rendering logic and data parsing.
- Maintain the foundational logic of previous tools while implementing a more efficient data structure (such as a hierarchical JSON) to represent the Kernel's responsibility distribution.
- Implement techniques such as hardware acceleration to eliminate the stuttering observed in previous iterations, ensuring a fluid interactive experience even at high depths of the directory tree.

3 Chronogram

Tasks	**M1**	**M2**	**M3**	**M4**	**M5**	**M6**	**M7**	**M8**	**M9**	**M10**
Literature review and study of current Treemap implementations	X	X	X							
Discussion and alignment of the optimized Treemap re-implementation architecture			X	X						
Implementation of the Treemap (as-is) with optimizations				X	X	X				
Evaluation of results and efficiency metrics obtained after re-implementation						X	X			
Write the monograph						X	X	X	X	
Monograph proofreading and corrections, if necessary										X

References

[1] Maxime André et al. "DENIM: Exploring Data Access in Microservices". In: *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 2025, pp. 900–904. DOI: 10.1109/ICSME64153.2025.00103.

[2] Brian Johnson and Ben Shneiderman. *Tree-Maps: A Space-Filling Approach to the Visualization of Hierarchical Information Structures*. 1991. URL: https://www.cs.umd.edu/~ben/papers/Johnson1991Tree.pdf.

[3] Michael Burch et al. "Visualizing work processes in software engineering with developer rivers". In: *2015 IEEE 3rd Working Conference on Software Visualization (VISSOFT)*. 2015, pp. 116–124. DOI: 10.1109/VISSOFT.2015.7332421.

[4] Manuel Hoffmann, Frank Nagle, Yanuo Zhou. *The Value of Open Source Software*. Jan. 1, 2024. URL: https://www.hbs.edu/ris/Publication%20Files/24-038_51f8444f-502c-4139-8bf2-56eb4b65c58a.pdf.

[5] Michael W. Godfrey and Qiang Tu. *Evolution in Open Source Software: A Case Study*. 2000. URL: https://turingmachine.org/opensource/papers/godfrey2000.pdf.

[6] Rafael Passos et al. "Streamlining Analyses on the Linux Kernel with DUKS". In: *2025 IEEE Working Conference on Software Visualization (VISSOFT)*. 2025, pp. 125–128. DOI: 10.1109/VISSOFT67405.2025.00025.

[7] Rafael Passos and Arthur Pilone Maia da Silva. *Dashboard for Unified Kernel Statistics*. URL: https://duks.rcpassos.me/.